

We'll go through:

1. 🧠 What an API is (and how it works)
2. 🧩 JavaScript basics you need
3. ⚙️ Making your first API call (GET request)
4. 📦 Working with JSON data
5. 🧱 Displaying API data on a web page
6. 📧 Sending data to an API (POST request)
7. 🚨 Handling errors & loading states
8. 🚀 Practice projects and next steps

1. 🧠 What is an API?

API = *Application Programming Interface*

It's a way for one program (like your webpage) to **communicate** with another (like a weather server).

For example:

- You → your browser (HTML/JS)
- Browser → API (like `https://api.weatherapi.com/...`)
- API → sends back data (usually JSON)
- JS → updates your web page with that data

Example response from an API:

```
{
  "city": "London",
  "temperature": 15,
  "condition": "Cloudy"
}
```

2. JavaScript Basics You Need

Here are the core JS concepts you should understand first:

★ Variables

```
let name = "Alice";  
const age = 25;
```

★ Functions

```
function greet() {  
  console.log("Hello!");  
}  
  
greet(); // runs the function
```

★ Objects and Arrays

```
let person = { name: "Alice", age: 25 };  
console.log(person.name); // "Alice"  
  
let fruits = ["apple", "banana", "mango"];  
console.log(fruits[1]); // "banana"
```

★ JSON

APIs often send data in **JSON (JavaScript Object Notation)**.

You convert JSON ↔ JavaScript like this:

```
let jsonString = '{"name":"Alice","age":25}';  
let data = JSON.parse(jsonString); // convert to JS object  
console.log(data.name);  
  
let backToJson = JSON.stringify(data); // convert JS object to JSON st
```

3. Making Your First API Call (GET Request)

Let's use the **Fetch API** (modern way).

```
<!DOCTYPE html>
<html>
<head>
  <title>Fetch API Example</title>
</head>
<body>
  <h2>Random Joke</h2>
  <button id="btn">Get Joke</button>
  <p id="output"></p>

  <script>
    document.getElementById("btn").addEventListener("click", getJoke);

    async function getJoke() {
      const response = await fetch("https://api.chucknorris.io/jokes/r
      const data = await response.json();
      document.getElementById("output").innerText = data.value;
    }
  </script>
</body>
</html>
```

How it works:

- `fetch()` sends an HTTP GET request.
- `await response.json()` converts the response to a JavaScript object.
- We then show the `data.value` (the joke text) on the page.

4. Working with JSON Data

If an API returns a list:

```
[
  {"id":1,"title":"Post One"},
```

```
{ "id": 2, "title": "Post Two" }  
]
```

You can loop through it in JS:

```
for (let post of data) {  
  console.log(post.title);  
}
```

5. 🧱 Displaying Data on a Web Page

Example: Fetch posts and show them as list items.

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Display API Data</title>  
</head>  
<body>  
  <h2>Posts</h2>  
  <ul id="posts"></ul>  
  
  <script>  
    async function loadPosts() {  
      const res = await fetch("https://jsonplaceholder.typicode.com/posts");  
      const data = await res.json();  
      const list = document.getElementById("posts");  
      list.innerHTML = ""; // clear previous items  
  
      data.slice(0, 5).forEach(post => {  
        const li = document.createElement("li");  
        li.textContent = post.title;  
        list.appendChild(li);  
      });  
    }  
  
    loadPosts();  
  </script>  
</body>  
</html>
```

6. 📧 Sending Data to an API (POST Request)

```
fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Hello", body: "World", userId: 1 })
})
  .then(res => res.json())
  .then(data => console.log("Created Post:", data))
  .catch(err => console.error("Error:", err));
```

7. 🚨 Error Handling & Loading

It's good to handle failed requests gracefully.

```
async function getData() {
  try {
    document.getElementById("status").innerText = "Loading...";
    const res = await fetch("https://api.chucknorris.io/jokes/random");
    if (!res.ok) throw new Error("Network error");
    const data = await res.json();
    document.getElementById("status").innerText = data.value;
  } catch (error) {
    document.getElementById("status").innerText = "Failed to fetch data";
  }
}
```

8. 🚀 Practice Projects

Try building these small apps:

1. **Joke Generator** — fetch new joke on button click.
2. **Weather App** — use [OpenWeather API](#).

3. **Quote of the Day** — display random quotes.
4. **GitHub Profile Finder** — show GitHub user info.
5. **To-Do App** — store and post tasks to an API.